

Manual Técnico

Sistema de Estacionamiento de Vehículos

IPC1 — Práctica 1

Descripción de arreglos, métodos, validaciones, cálculo de ruta y decisiones de implementación.

1. Introducción

El sistema está desarrollado en Java, exclusivamente de consola, y cumple con el requerimiento de usar únicamente arreglos nativos (unidimensionales y bidimensionales) para almacenar y gestionar la información — no se utiliza ArrayList, LinkedList, HashMap, HashSet, Stack, Queue ni ninguna otra colección dinámica.

El código está organizado en 7 clases, cada una con una responsabilidad específica, en lugar de resolver todo dentro del método main.

1.1 Arquitectura general

Clase	Responsabilidad
Practica1	Punto de entrada (main). Muestra el menú y delega cada opción a Estacionamiento.
Estacionamiento	Lógica de negocio: coordina Tablero, Caja y el arreglo de vehículos. Expone las 6 operaciones del menú.
Tablero	Representa la cuadrícula 10×10, la entrada/salida, y calcula la ruta más corta.
Vehiculo	Modelo de datos: placa, fila y columna de un vehículo estacionado.
Caja	Cobro de la tarifa fija y acumulación de ingresos.
ValidadorPlaca	Valida el formato de las placas (P###LLL).
Lector	Centraliza la lectura segura de datos desde consola (Scanner).

Estacionamiento no dibuja el tablero ni valida placas por sí misma: delega esas tareas a los objetos Tablero, Caja y ValidadorPlaca que tiene como atributos (composición). Esto permite que cada clase se modifique de forma independiente sin afectar a las demás.

2. Estructuras de datos (arreglos)

2.1 Tablero — `char[][]` celdas

El tablero completo se representa con una matriz de caracteres `char [11][11]` (los índices 1 a 10 se usan; el índice 0 se ignora, para que las coordenadas coincidan directamente con la numeración 1-10 del enunciado). Cada celda contiene uno de los siguientes símbolos:

- `'= '` — vía exterior
- `'E '` — entrada
- `'S '` — salida
- `'L '` — espacio interno libre
- `'A '` — espacio interno ocupado por un automóvil

Las filas y columnas 1 y 10 corresponden siempre a la vía exterior; las filas y columnas internas (donde se estacionan vehículos) son la 2 a la 9 del tablero completo, que se numeran del 1 al 8 de cara al usuario (fila interna = fila del tablero – 1).

2.2 Estacionamiento — `Vehiculo[]` vehiculos

Los vehículos estacionados se guardan en un arreglo nativo de objetos: `Vehiculo[] vehiculos = new Vehiculo[64]`. Un espacio libre en este arreglo se identifica con `vehiculos[i] == null`.

Se prefirió un arreglo de objetos `Vehiculo` en lugar de 3 arreglos paralelos (`placas[]`, `filas[]`, `columnas[]`) porque garantiza que la placa, la fila y la columna de un mismo vehículo viajen siempre juntas — con arreglos paralelos sería posible, por error, actualizar uno de los tres arreglos sin actualizar los otros dos, dejando los datos inconsistentes.

2.3 Tablero — `int[][]` perimetro

Para calcular la ruta más corta, el método `construirPerimetro()` genera un arreglo bidimensional `int[36][2]` con las 36 coordenadas (fila, columna) del borde exterior del tablero, ordenadas en sentido horario a partir de la esquina superior izquierda. Este arreglo se usa únicamente para el cálculo de la ruta y se reconstruye cada vez que se llama al método (no se guarda como atributo, ya que la entrada y la salida no cambian durante la ejecución del programa).

2.4 Otros arreglos

- `int[] generarPosicionBorde()` (en `Tablero`) devuelve un arreglo de 2 posiciones {fila, columna} — se usa como una forma simple de retornar dos valores desde un método sin necesitar una clase adicional.
- `int[] leerFilaColumnaLibre()` (en `Estacionamiento`) devuelve, de la misma forma, la fila y columna ya validadas donde se va a estacionar un vehículo.

3. Métodos principales por clase

3.1 Practica1

- `main(String[] args)`: configura la salida en UTF-8 (para que tildes y ñ se muestren bien en cualquier consola), crea el objeto Estacionamiento, y ejecuta el ciclo do-while del menú hasta que se elige la opción 7.
- `mostrarMenu()`: imprime las 7 opciones del menú principal.

3.2 Estacionamiento

- `ingresarVehiculo()`: valida capacidad máxima, pide placa (válida y no duplicada), fila/columna (válidas y libres), cobra la tarifa a través de Caja, y registra el vehículo en el arreglo y en el Tablero.
- `retirarVehiculo()`: busca la placa, libera la celda correspondiente en el Tablero, y pone en null la posición del arreglo.
- `mostrarEstacionamiento()`: delega la impresión del tablero a `Tablero.mostrar()`, pasándole el conteo de ocupados/libres.
- `buscarVehiculoPorPlaca()`: busca una placa en el arreglo y muestra su posición si existe.
- `mostrarRutaMasCorta()` / `mostrarIngresos()`: delegan directamente a `Tablero` y `Caja` respectivamente.
- Métodos privados de apoyo: `leerPlacaValidaNoDuplicada`, `leerPlacaValida`, `leerFilaColumnaLibre`, `posicionOcupada`, `buscarIndicePlaca`, `buscarIndiceLibre` — encapsulan la lógica de validación y búsqueda interna, sin exponerla fuera de la clase.

3.3 Tablero

- `inicializar()`: recorre las 10×10 celdas marcando el borde con '=' y el interior con 'L'.
- `generarEntradaSalida()` / `generarPosicionBorde()`: generan aleatoriamente la entrada y la salida sobre el borde, excluyendo esquinas (ver sección 5).
- `marcarOcupado(fila, columna)` / `marcarLibre(fila, columna)`: actualizan una celda interna (convirtiendo fila/columna 1-8 a coordenadas del tablero completo sumando 1).
- `mostrar(ocupados, libres)`: imprime el tablero completo con encabezados de fila/columna solo para las 8 posiciones internas.
- `mostrarRutaMasCorta()`, `construirPerimetro()`, `indiceEnPerimetro()`: implementan el cálculo de la ruta (ver sección 5).

3.4 Caja

- `cobrar()`: solicita el monto en un bucle hasta recibir un valor válido (no negativo, no menor a la tarifa), acumula el conteo de vehículos cobrados y el total recaudado, y devuelve el cambio.

- `mostrarIngresos()`: imprime el resumen de vehículos cobrados, tarifa y total recaudado.

3.5 ValidadorPlaca, Vehiculo y Lector

- `ValidadorPlaca.validar(placa)`: único método estático de la clase, valida el formato P###LLL carácter por carácter (ver sección 4).
- `Vehiculo`: clase modelo con constructor y getters (placa, fila, columna); no tiene setters porque un vehículo no cambia de posición una vez estacionado.
- `Lector`: centraliza `leerToken/leerEnteroSeguro/leerEnteroConEtiqueta/leerDecimalConEtiqueta`, todos con manejo de excepciones (`NumberFormatException`) para que una entrada no numérica nunca detenga el programa.

4. Validaciones implementadas

4.1 Formato de placa

ValidadorPlaca.validar() verifica, en orden:

- La placa no es null y tiene exactamente 7 caracteres.
- El primer carácter es 'P' mayúscula.
- Los caracteres en posiciones 1 a 3 son dígitos (Character.isDigit).
- Los caracteres en posiciones 4 a 6 son letras mayúsculas (Character.isUpperCase y Character.isLetter, para excluir minúsculas y símbolos).

4.2 Otras validaciones

- **Placa duplicada:** antes de ingresar un vehículo se verifica que la placa no exista ya en el arreglo.
- **Rango de fila/columna:** deben estar entre 1 y 8; fuera de ese rango se rechaza y se vuelve a pedir.
- **Espacio ocupado:** se verifica que ningún otro vehículo esté ya en esa fila/columna.
- **Estacionamiento lleno:** si ya hay 64 vehículos, no se permite ingresar uno más.
- **Monto de pago:** se rechaza si es negativo o menor a la tarifa (Q10.00); solo se acepta un monto igual o mayor.
- **Entradas no numéricas:** cualquier lectura de número (opción de menú, fila, columna, monto) está protegida con try/catch NumberFormatException, evitando que el programa termine de forma inesperada.
- **Fin de la entrada (EOF):** Lector.leerToken() verifica sc.hasNext() antes de leer; si la entrada estándar se cierra inesperadamente, el programa termina de forma controlada en lugar de lanzar una excepción sin manejar.

5. Cálculo de la ruta más corta

La entrada y la salida se ubican siempre sobre el borde del tablero completo (10×10), nunca en una esquina. Para generarlas, `generarPosicionBorde()` elige aleatoriamente uno de los 4 lados (superior, inferior, izquierdo, derecho) y luego una posición dentro de ese lado, excluyendo los extremos (columnas/filas 1 y 10), que son las esquinas.

Para comparar los dos recorridos posibles (horario y antihorario), `construirPerimetro()` genera un arreglo ordenado con las 36 celdas del borde exterior, recorridas en sentido horario a partir de la esquina superior izquierda (fila 1, columna 1):

```
1) Fila superior, de columna 1 a 10 → 2) Columna derecha, de fila 2 a 10 → 3) Fila inferior, de columna 9 a 1 → 4) Columna izquierda, de fila 9 a 2.
```

`indiceEnPerimetro()` busca en qué posición de ese arreglo (0 a 35) están la entrada y la salida. La distancia en sentido horario es la diferencia de índices (en módulo 36, para manejar el caso en que la salida está "antes" que la entrada en el arreglo):

```
horario = (idxSalida - idxEntrada + 36) % 36  
antihorario = 36 - horario
```

El sistema recomienda la ruta con menor distancia; si ambas son iguales, indica que cualquiera de las dos puede usarse.

Verificación con el ejemplo del enunciado: con entrada en (fila 1, columna 4) y salida en (fila 7, columna 10), el índice de la entrada en el perímetro es 3 y el de la salida es 15 (9 celdas de la fila superior restantes hasta la esquina, más 6 celdas bajando por la columna derecha). La distancia horaria es $(15 - 3 + 36) \% 36 = 12$, que coincide exactamente con el resultado del ejemplo del enunciado ("Ruta recomendada: sentido horario (12 posiciones)"), confirmando que el algoritmo está correctamente implementado.

6. Decisiones de implementación

División en 7 clases (una responsabilidad por clase)

Se separó el código en Vehiculo, ValidadorPlaca, Lector, Tablero, Caja, Estacionamiento y Practica1 en lugar de resolverlo todo en main(), siguiendo el principio de responsabilidad única: cada clase tiene una sola razón para cambiar, lo que facilita ubicar y corregir errores, y permite reutilizar código (por ejemplo, Lector puede usarse desde cualquier otra clase que necesite leer datos validados).

Vehiculo como objeto en vez de arreglos paralelos

Guardar placa, fila y columna dentro de un mismo objeto Vehiculo (en vez de tres arreglos String[], int[], int[] sincronizados por índice) evita el riesgo de que esos tres datos queden desincronizados por error.

Composición en Estacionamiento

Estacionamiento no hereda de Tablero ni de Caja; los tiene como atributos y delega en ellos. Esto refleja que un estacionamiento "tiene" un tablero y "tiene" una caja, no que "es" un tablero — es la relación correcta para este dominio.

Numeración visible solo para las 8 posiciones internas

En Tablero.mostrar(), las filas y columnas del borde (vía exterior) no llevan número, ya que no son posiciones válidas para estacionar; solo se numeran las 8 filas y 8 columnas internas, coincidiendo con los valores 1-8 que el usuario ingresa al registrar un vehículo, para evitar confusión entre la numeración interna y la del tablero completo.

Salida estándar forzada a UTF-8

Practica1.main() reconfigura System.out con StandardCharsets.UTF_8 al inicio, para que las tildes y la ñ se muestren correctamente sin importar la configuración regional del sistema operativo donde se ejecute el programa.

Lectura de datos siempre en bucle con validación

Cada dato solicitado al usuario (opción de menú, placa, fila, columna, monto) se lee dentro de un ciclo while(true) que solo retorna cuando el valor es válido, cumpliendo el requerimiento de que el programa nunca finalice de forma inesperada ni produzca errores de índice ante una entrada incorrecta.

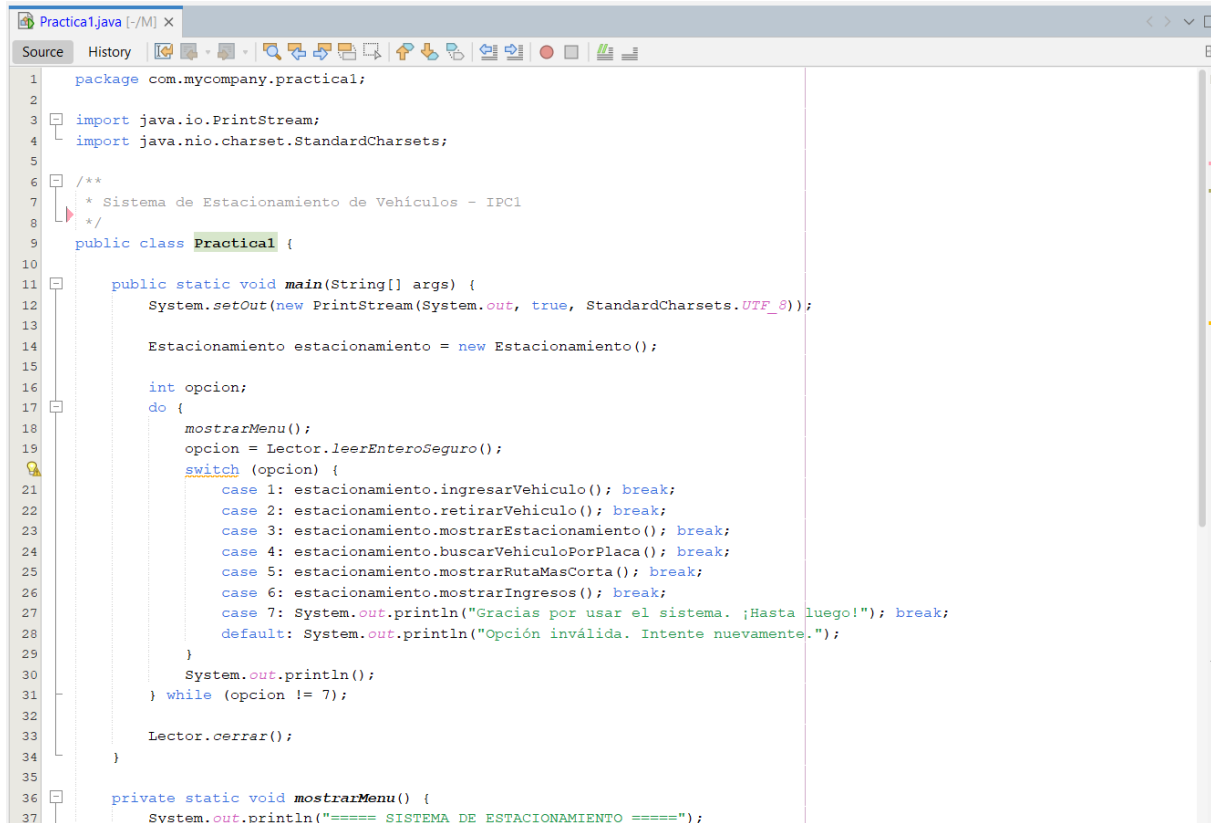
Sin colecciones dinámicas

Todos los datos variables (vehículos, coordenadas del perímetro, posiciones libres) se almacenan en arreglos nativos de tamaño fijo (o calculado una sola vez, como el perímetro de 36 celdas), en cumplimiento del requerimiento técnico de la práctica.

7. Apéndice — Código fuente

A continuación se incluye el código fuente completo de cada clase, organizado en el mismo orden en que se describieron en la sección 3.

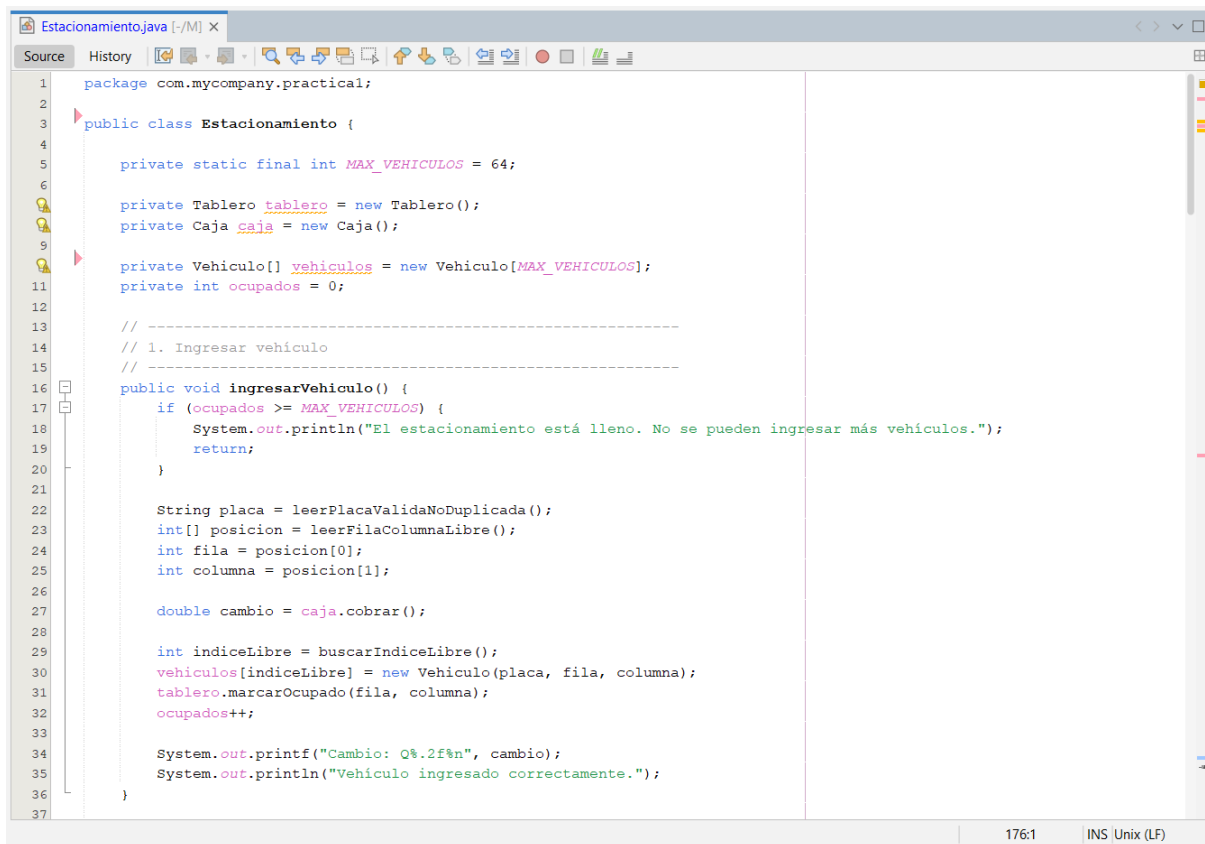
7.1 Practica1.java



```
1 package com.mycompany.practical;
2
3 import java.io.PrintStream;
4 import java.nio.charset.StandardCharsets;
5
6 /**
7  * Sistema de Estacionamiento de Vehículos - IPC1
8  */
9 public class Practical {
10
11     public static void main(String[] args) {
12         System.setOut(new PrintStream(System.out, true, StandardCharsets.UTF_8));
13
14         Estacionamiento estacionamiento = new Estacionamiento();
15
16         int opcion;
17         do {
18             mostrarMenu();
19             opcion = Lector.leerEnteroSeguro();
20             switch (opcion) {
21                 case 1: estacionamiento.ingresarVehiculo(); break;
22                 case 2: estacionamiento.retirarVehiculo(); break;
23                 case 3: estacionamiento.mostrarEstacionamiento(); break;
24                 case 4: estacionamiento.buscarVehiculoPorPlaca(); break;
25                 case 5: estacionamiento.mostrarRutaMasCorta(); break;
26                 case 6: estacionamiento.mostrarIngresos(); break;
27                 case 7: System.out.println("Gracias por usar el sistema. ¡Hasta luego!"); break;
28                 default: System.out.println("Opción inválida. Intente nuevamente.");
29             }
30             System.out.println();
31         } while (opcion != 7);
32
33         Lector.cerrar();
34     }
35
36     private static void mostrarMenu() {
37         System.out.println("==== SISTEMA DE ESTACIONAMIENTO =====");
```

```
Practical.java [-/M] x
Source History
13
14 Estacionamiento estacionamiento = new Estacionamiento();
15
16 int opcion;
17 do {
18     mostrarMenu();
19     opcion = Lector.leerEnteroSeguro();
20     switch (opcion) {
21         case 1: estacionamiento.ingresarVehiculo(); break;
22         case 2: estacionamiento.retirarVehiculo(); break;
23         case 3: estacionamiento.mostrarEstacionamiento(); break;
24         case 4: estacionamiento.buscarVehiculoPorPlaca(); break;
25         case 5: estacionamiento.mostrarRutaMasCorta(); break;
26         case 6: estacionamiento.mostrarIngresos(); break;
27         case 7: System.out.println("Gracias por usar el sistema. ¡Hasta luego!"); break;
28         default: System.out.println("Opción inválida. Intente nuevamente.");
29     }
30     System.out.println();
31 } while (opcion != 7);
32
33 Lector.cerrar();
34
35
36 private static void mostrarMenu() {
37     System.out.println("===== SISTEMA DE ESTACIONAMIENTO =====");
38     System.out.println("1. Ingresar vehículo");
39     System.out.println("2. Retirar vehículo");
40     System.out.println("3. Mostrar estacionamiento");
41     System.out.println("4. Buscar vehículo por placa");
42     System.out.println("5. Mostrar ruta más corta entre entrada y salida");
43     System.out.println("6. Mostrar ingresos");
44     System.out.println("7. Salir");
45     System.out.print("Seleccione una opción: ");
46 }
47
48
```

7.2 Estacionamiento.java



```
1 package com.mycompany.practical;
2
3 public class Estacionamiento {
4
5     private static final int MAX_VEHICULOS = 64;
6
7     private Tablero tablero = new Tablero();
8     private Caja caja = new Caja();
9
10    private Vehiculo[] vehiculos = new Vehiculo[MAX_VEHICULOS];
11    private int ocupados = 0;
12
13    // -----
14    // 1. Ingresar vehiculo
15    // -----
16    public void ingresarVehiculo() {
17        if (ocupados >= MAX_VEHICULOS) {
18            System.out.println("El estacionamiento está lleno. No se pueden ingresar más vehículos.");
19            return;
20        }
21
22        String placa = leerPlacaValidaNoDuplicada();
23        int[] posicion = leerFilaColumnaLibre();
24        int fila = posicion[0];
25        int columna = posicion[1];
26
27        double cambio = caja.cobrar();
28
29        int indiceLibre = buscarIndiceLibre();
30        vehiculos[indiceLibre] = new Vehiculo(placa, fila, columna);
31        tablero.marcarOcupado(fila, columna);
32        ocupados++;
33
34        System.out.printf("Cambio: Q%.2f\n", cambio);
35        System.out.println("Vehículo ingresado correctamente.");
36    }
37 }
```

176:1 INS Unix (LF)

```
Estacionamiento.java [-M] x
Source History
38 // -----
39 // 2. Retirar vehículo
40 // -----
41 public void retirarVehiculo() {
42     String placa = leerPlacaValida();
43     int indice = buscarIndicePlaca(placa);
44     if (indice == -1) {
45         System.out.println("No se encontró ningún vehículo con esa placa.");
46         return;
47     }
48     Vehiculo v = vehiculos[indice];
49     tablero.marcarLibre(v.getFila(), v.getColumna());
50     vehiculos[indice] = null;
51     ocupados--;
52
53     System.out.println("Vehículo retirado correctamente.");
54     System.out.println("Fila: " + v.getFila());
55     System.out.println("Columna: " + v.getColumna());
56 }
57
58 // -----
59 // 3. Mostrar estacionamiento
60 // -----
61 public void mostrarEstacionamiento() {
62     tablero.mostrar(ocupados, MAX_VEHICULOS - ocupados);
63 }
64
65 // -----
66 // 4. Buscar vehículo por placa
67 // -----
68 public void buscarVehiculoPorPlaca() {
69     String placa = leerPlacaValida();
70     int indice = buscarIndicePlaca(placa);
71     if (indice == -1) {
72         System.out.println("Vehículo no encontrado.");
73     }
74     return;
75 }
```

176:1 INS Unix (LF)

```
Estacionamiento.java [-M] x
Source History
80
81 // -----
82 // 5. Mostrar ruta más corta
83 // -----
84 public void mostrarRutaMasCorta() {
85     tablero.mostrarRutaMasCorta();
86 }
87
88 // -----
89 // 6. Mostrar ingresos
90 // -----
91 public void mostrarIngresos() {
92     caja.mostrarIngresos();
93 }
94
95 /** Pide una placa hasta que tenga formato válido y no esté ya registrada. */
96 private String leerPlacaValidaNoDuplicada() {
97     String placa;
98     while (true) {
99         System.out.print("Placa: ");
100         placa = Lector.leerToken();
101         if (!ValidadorPlaca.validar(placa)) {
102             System.out.println("Placa inválida. Formato requerido: P###LLL (ej. P401JZQ).");
103             continue;
104         }
105         if (buscarIndicePlaca(placa) != -1) {
106             System.out.println("Esa placa ya se encuentra registrada en el estacionamiento.");
107             continue;
108         }
109         return placa;
110     }
111 }
112
113 /** Pide una placa solo verificando el formato (para retirar/buscar). */
114 private String leerPlacaValida() {
115     String placa;
116     while (true) {
```

176:1 INS Unix (LF)

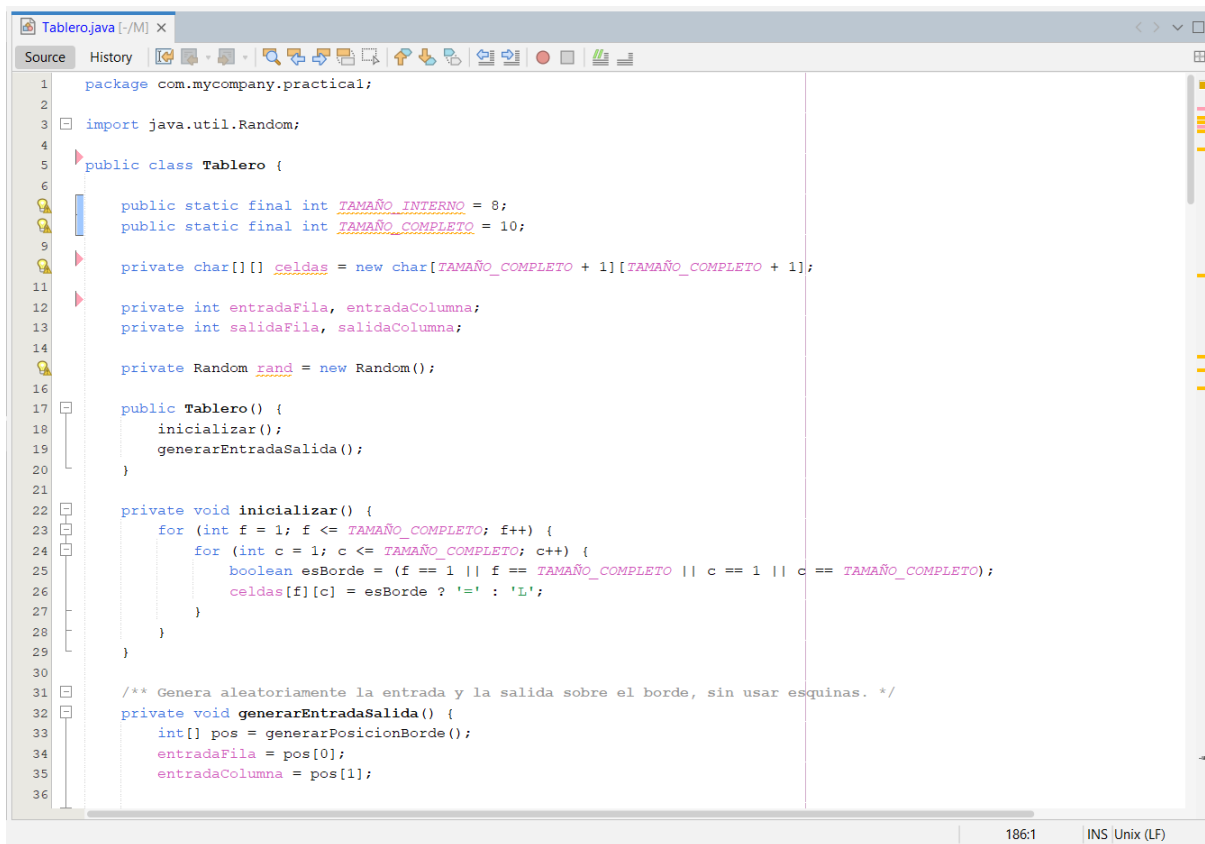
```
Estacionamiento.java [-/M] x
Source History
126
127 /** Pide fila y columna (1-8) validando rango y que el espacio esté libre. */
128 private int[] leerFilaColumnaLibre() {
129     int fila, columna;
130     while (true) {
131         fila = Lector.leerEnteroConEtiqueta("Fila: ");
132         if (fila < 1 || fila > Tablero.TAMAÑO_INTERNO) {
133             System.out.println("Fila fuera de rango. Debe estar entre 1 y 8.");
134             continue;
135         }
136         columna = Lector.leerEnteroConEtiqueta("Columna: ");
137         if (columna < 1 || columna > Tablero.TAMAÑO_INTERNO) {
138             System.out.println("Columna fuera de rango. Debe estar entre 1 y 8.");
139             continue;
140         }
141         if (posicionOcupada(fila, columna)) {
142             System.out.println("Ese espacio ya está ocupado. Elija otra posición.");
143             continue;
144         }
145         return new int[]{fila, columna};
146     }
147 }
148
149 private boolean posicionOcupada(int fila, int columna) {
150     for (int i = 0; i < MAX_VEHICULOS; i++) {
151         if (vehiculos[i] != null && vehiculos[i].getFila() == fila && vehiculos[i].getColumna() == columna) {
152             return true;
153         }
154     }
155     return false;
156 }
157
158 private int buscarIndicePlaca(String placa) {
159     for (int i = 0; i < MAX_VEHICULOS; i++) {
160         if (vehiculos[i] != null && vehiculos[i].getPlaca().equals(placa)) {
161             return i;
162         }
163     }
164 }
```

176:1 INS Unix (LF)

```
Estacionamiento.java [-/M] x
Source History
150 private boolean posicionOcupada(int fila, int columna) {
151     for (int i = 0; i < MAX_VEHICULOS; i++) {
152         if (vehiculos[i] != null && vehiculos[i].getFila() == fila && vehiculos[i].getColumna() == columna) {
153             return true;
154         }
155     }
156     return false;
157 }
158 private int buscarIndicePlaca(String placa) {
159     for (int i = 0; i < MAX_VEHICULOS; i++) {
160         if (vehiculos[i] != null && vehiculos[i].getPlaca().equals(placa)) {
161             return i;
162         }
163     }
164     return -1;
165 }
166 private int buscarIndiceLibre() {
167     for (int i = 0; i < MAX_VEHICULOS; i++) {
168         if (vehiculos[i] == null) {
169             return i;
170         }
171     }
172     return -1;
173 }
174 }
175
176
```

176:1 INS Unix (LF)

7.3 Tablero.java



```
1 package com.myccompany.practical;
2
3 import java.util.Random;
4
5 public class Tablero {
6
7     public static final int TAMAÑO_INTERNO = 8;
8     public static final int TAMAÑO_COMPLETO = 10;
9
10    private char[][] celdas = new char[TAMAÑO_COMPLETO + 1][TAMAÑO_COMPLETO + 1];
11
12    private int entradaFila, entradaColumna;
13    private int salidaFila, salidaColumna;
14
15    private Random rand = new Random();
16
17    public Tablero() {
18        inicializar();
19        generarEntradaSalida();
20    }
21
22    private void inicializar() {
23        for (int f = 1; f <= TAMAÑO_COMPLETO; f++) {
24            for (int c = 1; c <= TAMAÑO_COMPLETO; c++) {
25                boolean esBorde = (f == 1 || f == TAMAÑO_COMPLETO || c == 1 || c == TAMAÑO_COMPLETO);
26                celdas[f][c] = esBorde ? '=' : 'L';
27            }
28        }
29    }
30
31    /** Genera aleatoriamente la entrada y la salida sobre el borde, sin usar esquinas. */
32    private void generarEntradaSalida() {
33        int[] pos = generarPosicionBorde();
34        entradaFila = pos[0];
35        entradaColumna = pos[1];
36    }
```

186:1 INS Unix (LF)

```
Tablero.java [-/M] x
Source History
37 do {
38     pos = generarPosicionBorde();
39     salidaFila = pos[0];
40     salidaColumna = pos[1];
41 } while (salidaFila == entradaFila && salidaColumna == entradaColumna);
42
43 celdas[entradaFila][entradaColumna] = 'E';
44 celdas[salidaFila][salidaColumna] = 'S';
45
46
47 /** Devuelve una posición aleatoria sobre el borde del tablero, excluyendo las esquinas. */
48 private int[] generarPosicionBorde() {
49     int lado = rand.nextInt(4);
50     int fila, columna;
51     switch (lado) {
52     case 0:
53         fila = 1;
54         columna = 2 + rand.nextInt(TAMAÑO_COMPLETO - 2);
55         break;
56     case 1:
57         fila = TAMAÑO_COMPLETO;
58         columna = 2 + rand.nextInt(TAMAÑO_COMPLETO - 2);
59         break;
60     case 2:
61         columna = 1;
62         fila = 2 + rand.nextInt(TAMAÑO_COMPLETO - 2);
63         break;
64     default:
65         columna = TAMAÑO_COMPLETO;
66         fila = 2 + rand.nextInt(TAMAÑO_COMPLETO - 2);
67         break;
68     }
69     return new int[]{fila, columna};
70 }
71
72 /** Marca como ocupada (A) la celda correspondiente a una fila/columna interna (1-8). */
```

186:1 INS Unix (LF)

```
Tablero.java [-/M] x
Source History
72 /** Marca como ocupada (A) la celda correspondiente a una fila/columna interna (1-8). */
73 public void marcarOcupado(int fila, int columna) {
74     celdas[fila + 1][columna + 1] = 'A';
75 }
76
77 /** Marca como libre (L) la celda correspondiente a una fila/columna interna (1-8). */
78 public void marcarLibre(int fila, int columna) {
79     celdas[fila + 1][columna + 1] = 'L';
80 }
81
82 /** Imprime el tablero completo de 10x10 junto con el conteo de espacios. */
83 public void mostrar(int ocupados, int libres) {
84     System.out.println("E = Entrada      S = Salida      = = Vía exterior");
85     System.out.println("L = Lugar libre A = Automóvil");
86     System.out.println();
87
88     // Encabezado de columnas: solo se numeran las 8 columnas internas
89     // (1-8). Las columnas de la vía exterior se dejan en blanco.
90     System.out.print("    ");
91     for (int c = 1; c <= TAMAÑO_COMPLETO; c++) {
92         if (c == 1 || c == TAMAÑO_COMPLETO) {
93             System.out.print("    ");
94         } else {
95             System.out.print((c - 1) + " ");
96         }
97     }
98     System.out.println();
99
100     for (int f = 1; f <= TAMAÑO_COMPLETO; f++) {
101         if (f == 1 || f == TAMAÑO_COMPLETO) {
102             System.out.print("    ");
103         } else {
104             System.out.printf("%2d ", f - 1);
105         }
106         for (int c = 1; c <= TAMAÑO_COMPLETO; c++) {
107             System.out.print(celdas[f][c] + " ");
```

184:1 INS Unix (LF)


```
Tablero.java [-/M] x
Source History
109         System.out.println();
110     }
111
112     System.out.println();
113     System.out.println("Espacios ocupados: " + ocupados);
114     System.out.println("Espacios libres: " + libres);
115 }
116
117 /** Compara el recorrido horario y antihorario entre la entrada y la salida. */
118 public void mostrarRutaMasCorta() {
119     int[][] perimetro = construirPerimetro();
120     int total = perimetro.length;
121
122     int idxEntrada = indiceEnPerimetro(perimetro, entradaFila, entradaColumna);
123     int idxSalida = indiceEnPerimetro(perimetro, salidaFila, salidaColumna);
124
125     int horario = (idxSalida - idxEntrada + total) % total;
126     int antihorario = total - horario;
127
128     System.out.println("Entrada: fila " + entradaFila + ", columna " + entradaColumna);
129     System.out.println("Salida: fila " + salidaFila + ", columna " + salidaColumna);
130
131     if (horario == antihorario) {
132         System.out.println("Ambas rutas tienen la misma distancia (" + horario + " posiciones). Puede utilizar cualquier");
133     } else if (horario < antihorario) {
134         System.out.println("Ruta recomendada: sentido horario (" + horario + " posiciones)");
135         System.out.println("Ruta alterna: sentido antihorario (" + antihorario + " posiciones)");
136     } else {
137         System.out.println("Ruta recomendada: sentido antihorario (" + antihorario + " posiciones)");
138         System.out.println("Ruta alterna: sentido horario (" + horario + " posiciones)");
139     }
140 }
141
142 /** Construye, en orden horario, las coordenadas del borde exterior (36 celdas). */
143 private int[][] construirPerimetro() {
144     int total = 4 * (TAMAÑO_COMPLETO - 1); // 36 celdas para un borde de 10x10
145     int[][] perimetro = new int[total][2];
146     int idx = 0;
147
148     // Fila superior: de columna 1 a 10
149     for (int c = 1; c <= TAMAÑO_COMPLETO; c++) {
150         perimetro[idx][0] = 1;
151         perimetro[idx][1] = c;
152         idx++;
153     }
154     // Columna derecha: de fila 2 a 10
155     for (int f = 2; f <= TAMAÑO_COMPLETO; f++) {
156         perimetro[idx][0] = f;
157         perimetro[idx][1] = TAMAÑO_COMPLETO;
158         idx++;
159     }
160     // Fila inferior: de columna 9 a 1
161     for (int c = TAMAÑO_COMPLETO - 1; c >= 1; c--) {
162         perimetro[idx][0] = TAMAÑO_COMPLETO;
163         perimetro[idx][1] = c;
164         idx++;
165     }
166     // Columna izquierda: de fila 9 a 2
167     for (int f = TAMAÑO_COMPLETO - 1; f >= 2; f--) {
168         perimetro[idx][0] = f;
169         perimetro[idx][1] = 1;
170         idx++;
171     }
172     return perimetro;
173 }
174
175 private int indiceEnPerimetro(int[][] perimetro, int fila, int columna) {
176     for (int i = 0; i < perimetro.length; i++) {
177         if (perimetro[i][0] == fila && perimetro[i][1] == columna) {
178             return i;
179         }
180     }
181     return -1;
182 }
```

184:1

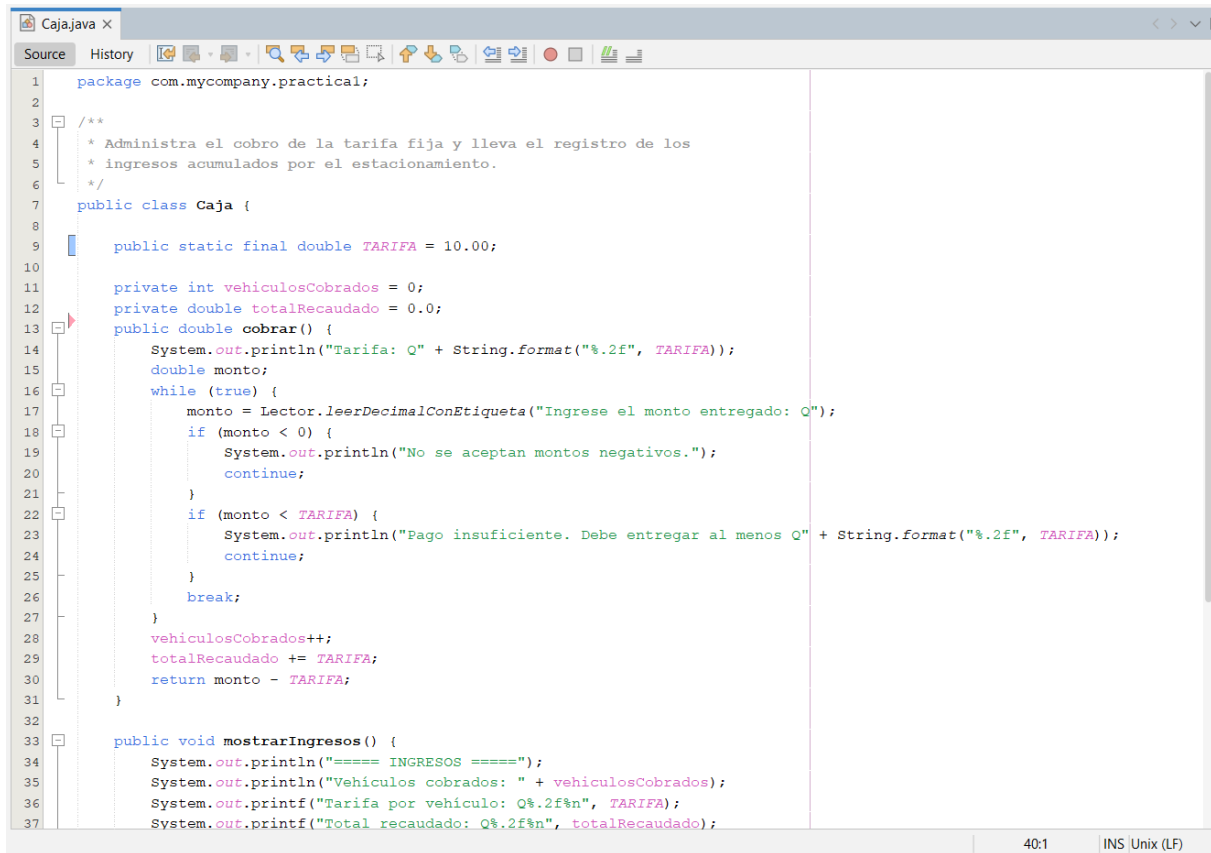
INS Unix (LF)

```
Tablero.java [-/M] x
Source History
142 /** Construye, en orden horario, las coordenadas del borde exterior (36 celdas). */
143 private int[][] construirPerimetro() {
144     int total = 4 * (TAMAÑO_COMPLETO - 1); // 36 celdas para un borde de 10x10
145     int[][] perimetro = new int[total][2];
146     int idx = 0;
147
148     // Fila superior: de columna 1 a 10
149     for (int c = 1; c <= TAMAÑO_COMPLETO; c++) {
150         perimetro[idx][0] = 1;
151         perimetro[idx][1] = c;
152         idx++;
153     }
154     // Columna derecha: de fila 2 a 10
155     for (int f = 2; f <= TAMAÑO_COMPLETO; f++) {
156         perimetro[idx][0] = f;
157         perimetro[idx][1] = TAMAÑO_COMPLETO;
158         idx++;
159     }
160     // Fila inferior: de columna 9 a 1
161     for (int c = TAMAÑO_COMPLETO - 1; c >= 1; c--) {
162         perimetro[idx][0] = TAMAÑO_COMPLETO;
163         perimetro[idx][1] = c;
164         idx++;
165     }
166     // Columna izquierda: de fila 9 a 2
167     for (int f = TAMAÑO_COMPLETO - 1; f >= 2; f--) {
168         perimetro[idx][0] = f;
169         perimetro[idx][1] = 1;
170         idx++;
171     }
172     return perimetro;
173 }
174
175 private int indiceEnPerimetro(int[][] perimetro, int fila, int columna) {
176     for (int i = 0; i < perimetro.length; i++) {
177         if (perimetro[i][0] == fila && perimetro[i][1] == columna) {
178             return i;
179         }
180     }
181     return -1;
182 }
```

184:1

INS Unix (LF)

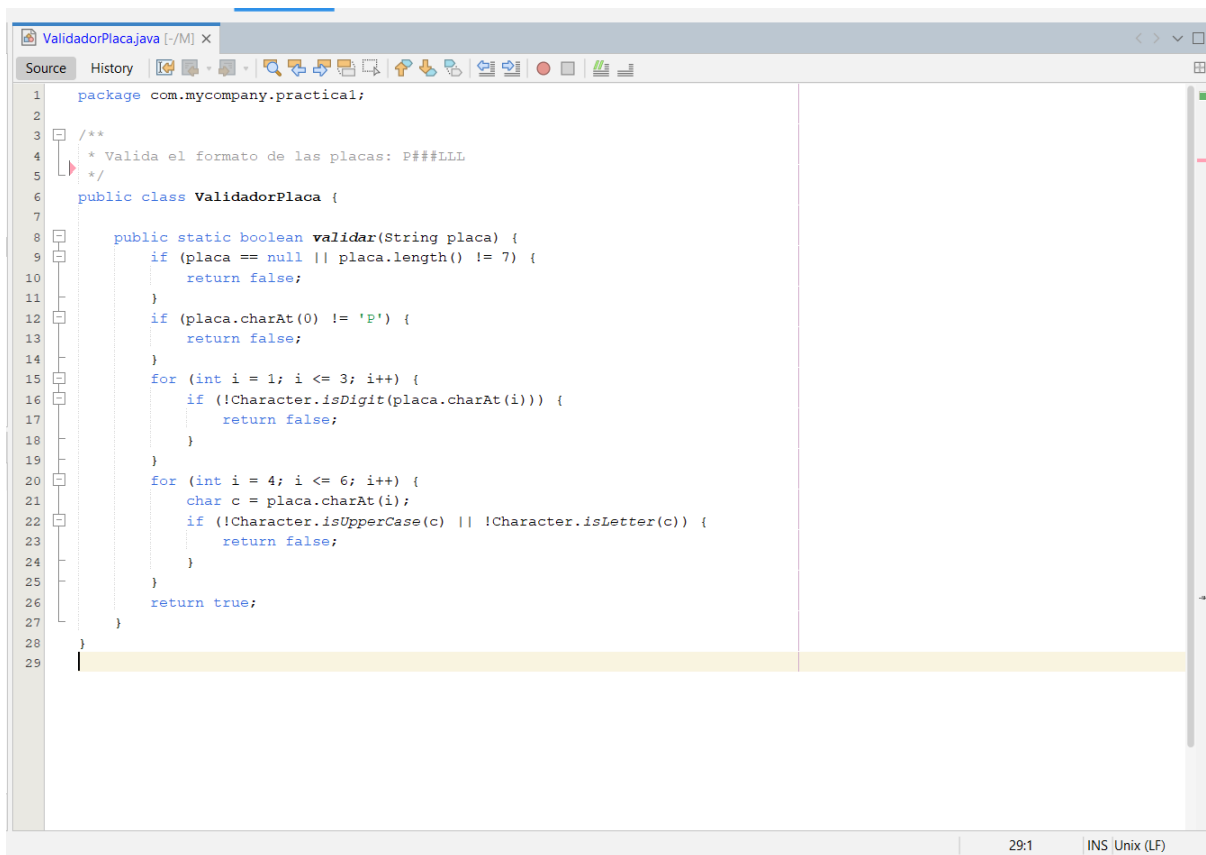
7.4 Caja.java



```
1 package com.mycompany.practical;
2
3 /**
4  * Administra el cobro de la tarifa fija y lleva el registro de los
5  * ingresos acumulados por el estacionamiento.
6  */
7 public class Caja {
8
9     public static final double TARIFA = 10.00;
10
11     private int vehiculosCobrados = 0;
12     private double totalRecaudado = 0.0;
13
14     public double cobrar() {
15         System.out.println("Tarifa: Q" + String.format("%.2f", TARIFA));
16         double monto;
17         while (true) {
18             monto = Lector.leerDecimalConEtiqueta("Ingrese el monto entregado: Q");
19             if (monto < 0) {
20                 System.out.println("No se aceptan montos negativos.");
21                 continue;
22             }
23             if (monto < TARIFA) {
24                 System.out.println("Pago insuficiente. Debe entregar al menos Q" + String.format("%.2f", TARIFA));
25                 continue;
26             }
27             break;
28         }
29         vehiculosCobrados++;
30         totalRecaudado += TARIFA;
31         return monto - TARIFA;
32     }
33
34     public void mostrarIngresos() {
35         System.out.println("==== INGRESOS =====");
36         System.out.println("Vehículos cobrados: " + vehiculosCobrados);
37         System.out.printf("Tarifa por vehículo: Q%.2f\n", TARIFA);
38         System.out.printf("Total recaudado: Q%.2f\n", totalRecaudado);
39     }
40 }
```

40:1 INS Unix (LF)

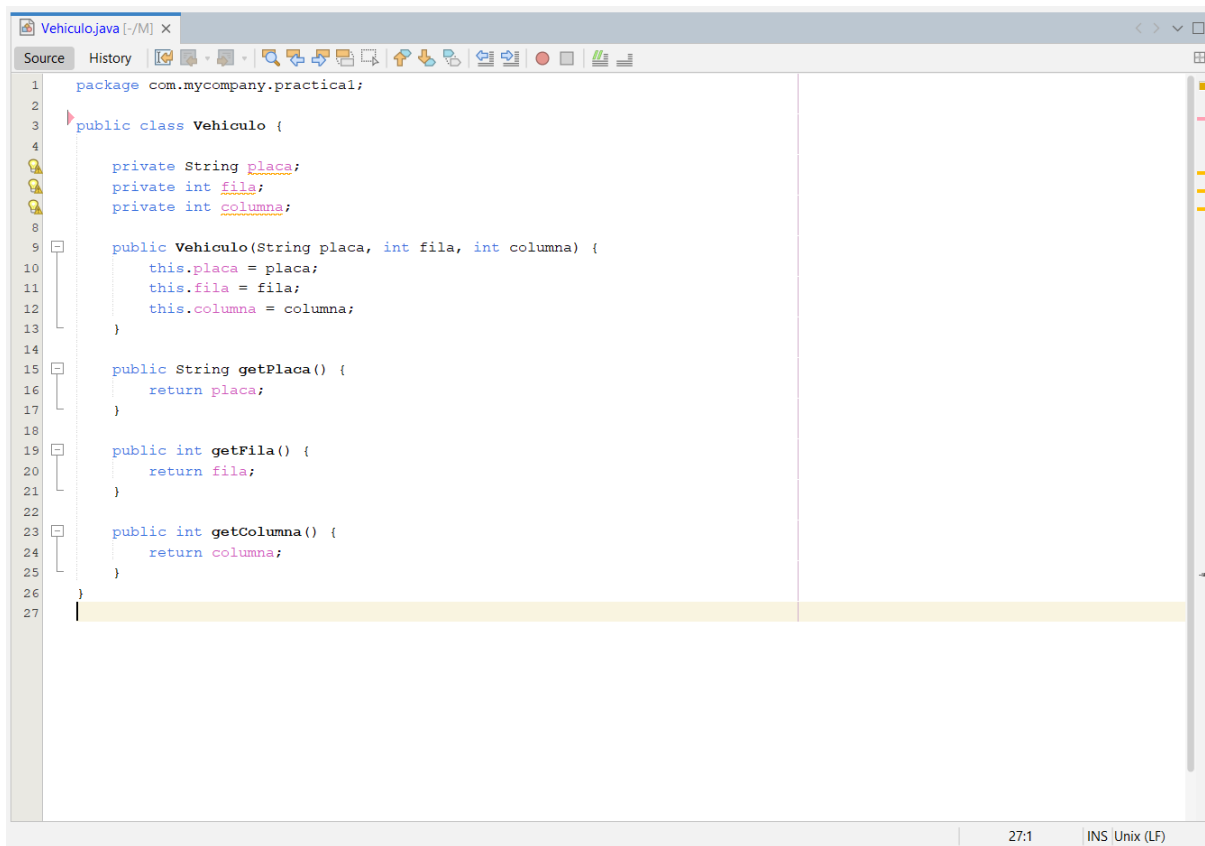
7.5 ValidadorPlaca.java



```
1 package com.mycompany.practical;
2
3 /**
4  * Valida el formato de las placas: P###LLL
5  */
6 public class ValidadorPlaca {
7
8     public static boolean validar(String placa) {
9         if (placa == null || placa.length() != 7) {
10             return false;
11         }
12         if (placa.charAt(0) != 'P') {
13             return false;
14         }
15         for (int i = 1; i <= 3; i++) {
16             if (!Character.isDigit(placa.charAt(i))) {
17                 return false;
18             }
19         }
20         for (int i = 4; i <= 6; i++) {
21             char c = placa.charAt(i);
22             if (!Character.isUpperCase(c) || !Character.isLetter(c)) {
23                 return false;
24             }
25         }
26         return true;
27     }
28 }
29
```

29:1 INS Unix (LF)

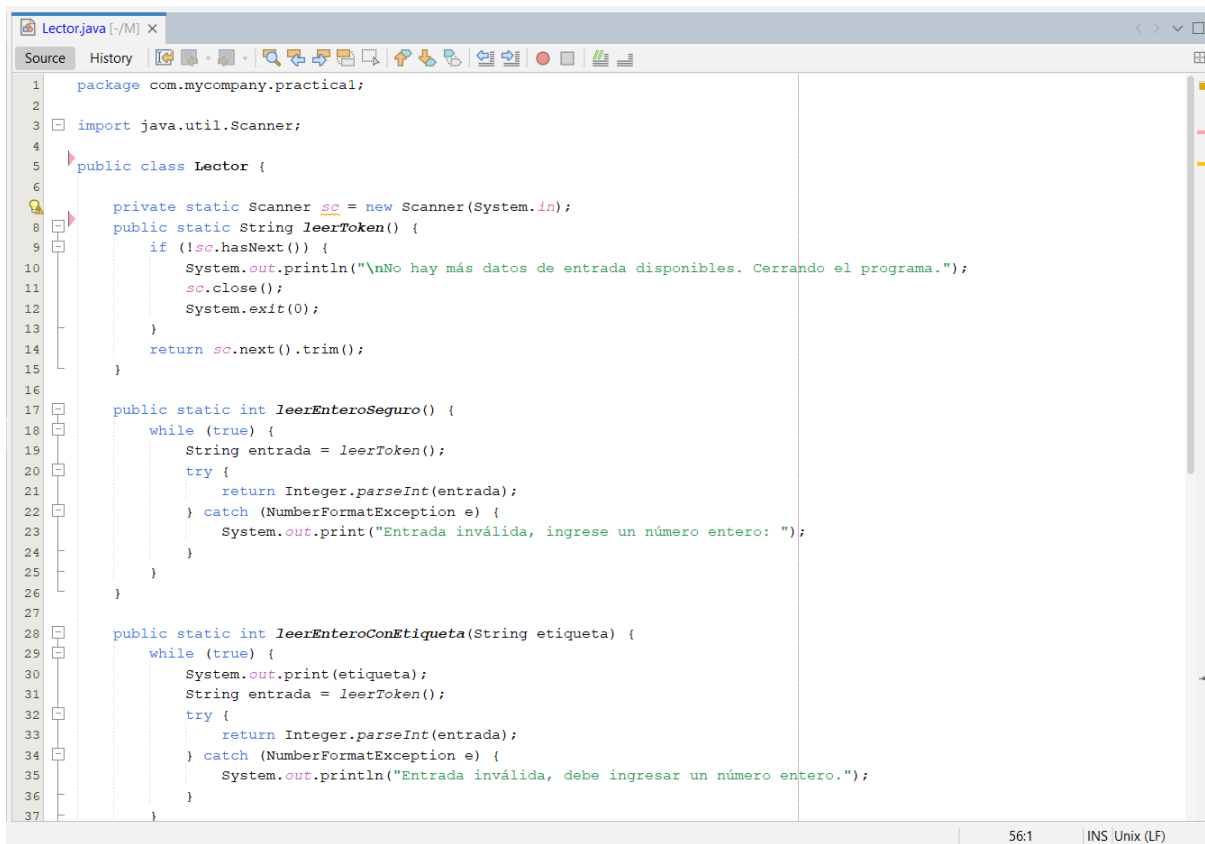
7.6 Vehiculo.java



```
1 package com.mycompany.practical;
2
3 public class Vehiculo {
4
5     private String placa;
6     private int fila;
7     private int columna;
8
9     public Vehiculo(String placa, int fila, int columna) {
10         this.placa = placa;
11         this.fila = fila;
12         this.columna = columna;
13     }
14
15     public String getPlaca() {
16         return placa;
17     }
18
19     public int getFila() {
20         return fila;
21     }
22
23     public int getColumna() {
24         return columna;
25     }
26 }
27
```

27:1 INS Unix (LF)

7.7 Lector.java



```
1 package com.mycompany.practical;
2
3 import java.util.Scanner;
4
5 public class Lector {
6
7     private static Scanner sc = new Scanner(System.in);
8     public static String leerToken() {
9         if (!sc.hasNext()) {
10             System.out.println("\nNo hay más datos de entrada disponibles. Cerrando el programa.");
11             sc.close();
12             System.exit(0);
13         }
14         return sc.next().trim();
15     }
16
17     public static int leerEnteroSeguro() {
18         while (true) {
19             String entrada = leerToken();
20             try {
21                 return Integer.parseInt(entrada);
22             } catch (NumberFormatException e) {
23                 System.out.print("Entrada inválida, ingrese un número entero: ");
24             }
25         }
26     }
27
28     public static int leerEnteroConEtiqueta(String etiqueta) {
29         while (true) {
30             System.out.print(etiqueta);
31             String entrada = leerToken();
32             try {
33                 return Integer.parseInt(entrada);
34             } catch (NumberFormatException e) {
35                 System.out.println("Entrada inválida, debe ingresar un número entero.");
36             }
37         }
38     }
39 }
```

56:1 INS Unix (LF)

```
Lector.java [-/M] x
Source History
32 try {
33     return Integer.parseInt(entrada);
34 } catch (NumberFormatException e) {
35     System.out.println("Entrada inválida, debe ingresar un número entero.");
36 }
37 }
38 }
39
40 public static double leerDecimalConEtiqueta(String etiqueta) {
41     while (true) {
42         System.out.print(etiqueta);
43         String entrada = leerToken();
44         try {
45             return Double.parseDouble(entrada);
46         } catch (NumberFormatException e) {
47             System.out.println("Entrada inválida, debe ingresar un número (ejemplo: 20.00).");
48         }
49     }
50 }
51
52 public static void cerrar() {
53     sc.close();
54 }
55 }
56
```

56:1 INS Unix (LF)